



Unidad 3

Clases abstractas e interfaces

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026



La pre clase, en dos preguntas



- La clase pasada quedó armada la jerarquía `Pago`, con `PagoDebito`, `PagoCredito` y `PagoQR`. Lo común, escrito una sola vez.
- ¿Qué significa el objeto que devuelve `new Pago(1, venta)`? ¿Por qué compila algo que en el local no existe?
- Cada medio de pago tiene un IVA distinto. ¿Donde va ese cálculo, y quien avisa si una subclase se lo olvida?

Las dos preguntas son el mismo agujero: falta poder decir que una clase esta incompleta a proposito.

Agujero 1: se puede instanciar lo que no existe



```
Pago pago = new Pago(1, venta);  
  
// compila, corre, y no es un debito,  
// ni un credito, ni un QR.
```

Pago se escribió para ser heredada, no para ser instanciada. Y en ningún lugar del código se especifica.

Agujero 2: nadie obliga a completar



```
public class Pago {  
  
    // Si se declara con un cuerpo cualquiera,  
    // el que se olvide de sobrescribirlo hereda  
    // un cálculo erróneo, y no salta error.  
    public double montoAcreditado() {  
        return this.getMonto();  
    }  
}
```

El compilador no avisa: la subclase hereda un método que existe.

abstract: incompleta a proposito



```
public abstract class Pago {  
  
    // Declara que el cálculo existe. No lo escribe:  
    // Pago le falta info para implementarlo.  
    public abstract double montoAcreditado();  
  
    public void mostrarComprobante() {  
        System.out.println("Operacion nro " + this.numero);  
        System.out.println("Se acreditan: $" + this.montoAcreditado());  
    }  
}
```

`new Pago(...)` deja de compilar, y toda subclase queda obligada a escribir el calculo.

mostrarComprobante() es un método concreto

que llama a un método del que Pago no tiene una sola línea escrita.

Las reglas de abstract



Lo que obliga

- Una clase con un método abstracto tiene que declararse `abstract`.
- La subclase que no los implementa todos tiene que ser `abstract` también.
- `new` sobre una clase abstracta no compila.

Lo que sigue igual

- Puede ser `abstract` sin ningún método abstracto, sólo para prohibir el `new`.
- Si tiene constructor: lo llama el `super(...)` del hijo.
- Tiene atributos y métodos concretos como cualquier clase.

Sigue siendo una clase. Lo único que cambia es que no se la puede instanciar.

Un requerimiento que la herencia no modela



- Ahora se cobra también en efectivo, y hay que poder anular una operacion del dia.
- El débito se anula pidiendo el reintegro a la tarjeta. El QR, avisando a la billetera virtual. El efectivo no se anula desde el sistema.
- Poner `anular()` en `Pago`? `PagoEfectivo` hereda una capacidad que no tiene.
- Una clase intermedia `PagoElectronico`? Una `Venta` también se anula, y no es un `Pago`.

Poder anularse no dice que es el objeto. Dice que sabe hacer, y cruza la jerarquia en diagonal.

interface: el contrato puro



```
public interface Anulable {  
  
    // public static final, aunque no se escriba  
    int DIAS_PARA_ANULAR = 30;  
  
    // public abstract, aunque no se escriba  
    boolean anular();  
  
    boolean estaAnulado();  
}
```

Un contrato con cláusulas privadas no le sirve a nadie: la interfaz existe para exponer.

extends e implements conviven



```
public class PagoQR extends Pago implements Anulable {  
  
    @Override  
    public boolean anular() {  
        if (this.anulado || this.diasPasados() > DIAS_PARA_ANULAR) {  
            return false;  
        }  
        this.anulado = true;  
        return true;  
    }  
}  
  
public class Venta implements Anulable { ... }
```

Un solo padre, todos los contratos que hagan falta. Venta cumple el mismo sin compartir ancestro.

Cual de los dos



Clase abstracta: es un

- Un pago con QR **es** un pago.
- `extends Pago`, una sola.
- Aporta atributos y código ya escrito, mas los huecos obligatorios.
- Varias clases parecidas comparten estado y código.

Interfaz: sabe hacer

- Un pago con QR **sabe** anularse.
- `implements Anulable`, todas las que hagan falta.
- Aporta solo el contrato, ninguna implementación.
- La capacidad aparece en clases sin parentesco.

La pregunta que las separa no es técnica, es de significado. La misma que decidía si correspondía heredar.

Programar contra el contrato



```
List<String> nombres = new ArrayList<>();  
  
// Recorre pagos con QR, débitos y ventas sin distinguirlos.  
// No se toca cuando aparezca la próxima clase que se anule.  
public static int anularTodo(Anulable[] operaciones) { ... }
```

Que el mismo llamado ejecute código distinto según quien está del otro lado tiene nombre: polimorfismo, la clase que viene.

La actividad de hoy



- Se sigue con la tienda de la clase pasada, con la jerarquía `Producto` ya resuelta.
- **Parte 1:** encontrar los dos problemas. Un artículo que no es de ningún tipo, y el IVA del 21% cobrado a los tres productos.
- **Parte 2:** `Producto` pasa a abstracta y `getPrecioFinal()` a abstracto. Cada subclase escribe el suyo.
- **Parte 3:** la interfaz `Descontable`, que cumplen `Libro`, `Alimento` y `Suscripcion`, que no es un `Producto`.

Para cerrar: meter objeto notebook en el arreglo de `Descontable` y leer el error. Esa línea es el argumento de la clase.